DEVSECURE PLATFORM

Technical Architecture & Autonomous Remediation Engine

Version:	4.0
Date:	February 2026
Classification:	Technical Architecture Document
Audience:	Technical Directors, AI Angel Investors, Architecture Engineers

1. Vision & Problem Statement

The modern software supply chain is facing a vulnerability overload crisis. As development velocity increases, the volume of security alerts generated by traditional tooling has become unmanageable. DevSecOps teams suffer from acute alert fatigue, drowning in thousands of findings that lack context, prioritization, or a clear path to remediation.

The fundamental gap in the market is not detection capability—scanners are plentiful—but actionable intelligence. Teams struggle to answer the critical question: "Of these 5,000 alerts, which 5 must we fix today to actually reduce risk?"

DevSecure's Promise: To transition application security from a blocking gate to an intelligent enabler. By providing deterministic, multi-source risk prioritization and AI-powered remediation, DevSecure transforms raw data into a ranked, executable action plan.

2. Market Failure of Traditional SAST

Traditional Static Application Security Testing (SAST) tools have failed to keep pace with modern engineering needs due to inherent structural flaws:

- **CVSS-Only Prioritization Blindness:** Relying solely on the Common Vulnerability Scoring System (CVSS) ignores the context of the

environment. A high-severity theoretical vulnerability that is unreachable or unexploitable in reality should not block a release.

- **Scanner Severity without Context:** Vendors often assign "Critical" severity to generic issues to protect themselves from liability, transferring the triage burden to the customer.
- **Flat Lists vs. Ranked Intelligence:** Providing an unranked list of 10,000 issues is effectively the same as providing zero issues. Without intelligent ranking, remediation becomes arbitrary.
- **Missing Exploitation Reality:** Most tools fail to correlate findings with active threat intelligence (like CISA KEV or EPSS), leading to wasted effort on non-threatening bugs.
- **The "Fix Everything" Impossibility:** The premise of zero vulnerabilities is mathematically impossible in modern distributed systems. The goal must be risk management, not bug elimination.

3. DevSecure Platform Overview

DevSecure is an All-in-One Application Security Posture Management (ASPM) SaaS platform. It is designed specifically for developers and DevSecOps teams who require speed and precision.

Core Capabilities

- AI-powered SAST & Secrets Detection
- Dynamic Application Security Testing (DAST)
- Software Bill of Materials (SBOM) & SCA
- Infrastructure as Code (IaC) Security
- Mobile Security & Malware Detection
- Vulnerability Management

Key Differentiator: The Intelligent Prioritization Engine (RPS), which serves as the decision-making spine of the platform.

Architecture: The platform is cloud-native, running on a k3s cluster with a high-performance Go Router for request handling and traffic management, secured behind Cloudflare's edge network.

> *Note: See Figure 1 (High-Level System Architecture) in presentation deck for component visualization.*

4. Multi-Source Intelligence Architecture

DevSecure employs an "Intelligence Fusion" strategy rather than simple data enrichment. We actively correlate data from nine distinct authoritative sources to form a high-fidelity risk signal.

Source	Primary Value Contribution
NVD	Baseline CVSS scores and attack vectors.
MITRE	Canonical CVE metadata and classification.
OSV	Ecosystem-specific vulnerability confirmation.
GHSA	GitHub Advisory presence and developer context.
ExploitDB	Evidence of exploit code availability.
EPSS	Probability of exploitation in the next 30 days.
CISA KEV	Confirmation of active exploitation in the wild.
CVEfixes/MoreFixes	Patch availability, code diffs, and remediation patterns.
Internal AI	LLM-derived CWE confidence and logic analysis.

5. BigQuery + dbt Medallion Design

The data platform follows a strict Medallion Architecture implemented on Google BigQuery, orchestrated by dbt (data build tool) for governance, lineage, and testing.

- **Bronze (Raw):** Immutable landing zone for ingestion. No transformations. Preserves history.
- **Silver (Curated):** Cleaned, normalized, and deduped data. Enforces schema contracts and logic curation.
- **Gold (AI/Serving):** Aggregated business-level tables optimized for ML training and API consumption.

6. Raw Ingestion (Bronze Layer)

The Bronze layer prioritizes write speed and immutability. All external data lands here first. Code artifacts (diffs, source files) are stored in Google Cloud Storage (GCS) to keep BigQuery lean, with URIs stored in tables.

Primary Schemas: `morefixes_raw_commits`, `morefixes_raw_metadata`, `morefixes_raw_fixes`, `ghsa_raw_advisories`, `freshfix_raw_refs`,

`freshfix_raw_pairs`.

This layer enables full replayability. If our parsing logic improves, we can re-process the raw JSON payloads without data loss.

7. Silver Canonical Models (Silver Layer)

The Silver layer is the "Source of Truth." Here, we standardize disparate data sources into a unified schema. Crucially, this is where quality filtering occurs.

Key Transformations:

- **Deduping:** Resolving conflicting records for the same CVE.
- **CWE Filtering:** We specifically filter for "Logic/CWE-like changes," excluding pure dependency version bumps which do not teach the AI model how to fix code.
- **Provenance Scoring:** Assigning trust scores to data based on source reliability (e.g., Official Maintainer vs. Community Contribution).

The core canonical table is `d_patch`, a unioned dimension table carrying the consolidated patch intelligence.

8. Gold/AI Contracts (Gold Layer)

The Gold layer is optimized for high-performance reading by APIs and ML pipelines. It abstracts the complexity of the Silver layer into clean contracts.

- `v_ai_train_patch_pairs`: Highly curated code pairs for LLM fine-tuning.
- `v_patch_summary_by_cve`: Lightweight metadata view for the RPS API, containing pointers to GCS artifacts rather than blobs.

9. RPS Risk Engine: DEEP DIVE

9.1 Why RPS Exists

Traditional vulnerability scoring is broken. A CVSS 9.8 vulnerability in a library that is never called by the application is practically irrelevant compared to a CVSS 7.5 vulnerability that is actively being exploited in the wild (CISA KEV). RPS (Risk Prioritization Score) exists to bridge this gap, turning theoretical severity into actionable business risk.

9.2 RPS Philosophy: Deterministic, Not a Guess

RPS_Risk is Deterministic, Not a Model Hallucination. Unlike black-box AI models that might "guess" a priority, RPS_Risk is a transparent, auditable algorithm with explicit mathematical foundations. Every score can be traced back to its constituent signals from authoritative threat intelligence sources.

Unbounded by Design: Starting in v2.6.3, the platform made a critical architectural decision to keep RPS_Risk unbounded (no caps at 100 or any arbitrary limit). This preserves analytical headroom and prevents hidden biases. When multiple threat signals align (high CVSS + high EPSS + KEV = true), the uncapped score correctly reflects the compounding risk.

9.3 RPS Scoring Formula (Detailed)

The RPS formula produces an unbounded risk value to preserve analytical headroom. It uses a multiplicative weighting of three core factors:

```
RPS_Risk = (0.4 × CVSS_curated/10) × (0.3 × EPSS_percentile) × (0.3 × 𝟙_CISA_KEV) × 100
```

- **CVSS Curated (Weight: 0.4):** Normalized CVSS score (0-10 scale, divided by 10). Uses explicit precedence: CVSS v3.1 → v3.0 → v4.0 → v2.0 → OSV severity.
- **EPSS Percentile (Weight: 0.3):** Exploit Prediction Scoring System percentile (0-100 scale). Provides forward-looking likelihood based on active exploitation patterns.
- **CISA KEV Indicator (Weight: 0.3):** Binary multiplier for Known Exploited Vulnerabilities catalog. When true (1), confirms active exploitation. When false, dampens the score.

The multiplicative nature is intentional. If any factor is low (especially KEV = 0), the entire score is appropriately dampened. Conversely, when all three signals align, the score reflects genuine critical risk.

9.4 RPS_Risk vs RPS_Score: Architecture Clarification

Aspect	RPS_Risk (Engine)	RPS_Score (Presentation)
Layer	Silver (devsec_curated)	Gold (devsec_ai) / API
Field Name	f_vulnerability.rps_risk	v_unified_risk_score.rps_score
Bounded?	Unbounded (can exceed 100)	Optionally normalized to 0-100 for UI
Purpose	Analytical precision, ML input	User-friendly display, dashboards
Computation	Computed once (single source of truth)	Pure projection (no recalculation)

9.5 Score Interpretation Bands (Presentation Layer)

- 90-100 (CRITICAL): Active exploitation confirmed (KEV) or imminent threat. Fix immediately.
- 70-89 (HIGH): High probability of exploitation with significant impact. Fix this sprint.
- 50-69 (MEDIUM): Weaponizable but dormant. Prioritize in backlog.
- 30-49 (LOW): Limited exploitation potential or low impact. Defer or accept risk.
- 0-29 (INFORMATIONAL): Theoretical risk only. Guidance or documentation fixes.

9.6 Why RPS_Risk ≠ CVSS

The power of RPS_Risk lies in its multiplicative integration of exploitation reality. A CVSS 6.0 vulnerability with KEV can (and should) outrank a CVSS 10.0 vulnerability without exploitation evidence. This redirects engineering effort toward real threats, not theoretical maximums.

10. Prioritization API Design

The RPS engine is exposed via a high-performance REST API running on Google Cloud Run. The API utilizes heavy caching to ensure sub-100ms response times for common queries, with a batch endpoint available for scanning thousands of dependencies simultaneously.

11. SAST Detection Layer - CCN-Aware Hybrid Approach

DevSecure utilizes a hybrid detection strategy optimized for both speed and semantic depth. We route file scanning based on the Cyclomatic Complexity Number (CCN) of the target code.

11.1 Detection Logic

- **Low Complexity (CCN ≤ 15):** Traffic is routed to **OpenGrep/Semgrep**. These deterministic engines provide millisecond-latency pattern matching for standard vulnerabilities.
- **High Complexity (CCN > 15):** Traffic is routed to **Qwen3-Coder**. This LLM-based detection engine performs deep semantic analysis to identify subtle logic flaws that traditional regex-based scanners miss.

11.2 Deterministic Corroboration Invariant

A critical architectural invariant is that LLM-detected findings (from Qwen3-Coder) are never accepted blindly. They require deterministic corroboration. The detection engine must be able to generate a "proof of existence" (e.g., a synthesized test case or a specific data-flow path) that can be verified deterministically.

12. Classification Plane - Qwen2.5-Coder-1.5B Advisory SLM

(ADR Amendment A1: The legacy 3-tier classification system using DeBERTa and GraphCodeBERT has been deprecated to reduce latency.)

The Classification Plane now utilizes a single-pass **Qwen2.5-Coder-1.5B (Advisory SLM)**. This lightweight model is optimized for extreme speed and strict schema adherence.

12.1 Operational Flow

1. The finding (code snippet + metadata) is sent to the SLM.
2. The SLM outputs a strict JSON object containing:
   - **CWE ID:** The specific Common Weakness Enumeration.
   - **Relative Confidence:** A probability score (0.0 - 1.0).
   - **Complexity Flags:** A 26-dimensional feature vector (see Section 13).

Fail-Closed Classification: If the SLM indicates uncertainty or fails to generate a valid JSON schema, the system defaults to **Lane L4 (Manual Routing)**. There is no fallback to human review tiers; the system strictly fails closed to the safest possible lane.

13. Remediation Control Plane - Lane-Based Routing

Routing is no longer determined by simple "Fix Now/Template" paths. Instead, the Remediation Control Plane evaluates a **26-dimensional complexity vector** to assign every finding to one of four specific remediation lanes.

13.1 Complexity Dimensions

The routing vector includes:

- **Code Context Depth:** How much surrounding code is needed to understand the fix.
- **Test Coverage:** Availability of existing tests.

- **Blast Radius:** Potential impact on dependent modules.
- **Change Frequency:** Volatility of the target file.
- **Dataflow Complexity:** Depth of variable propagation.

13.2 Routing Lanes

Lane	Description	Typical Target
L1 (Trivial)	Simple, atomic patterns. Minimal context required.	Missing headers, weak ciphers.
L2 (Localized)	Single-file changes with localized impact.	Input validation, resource cleanup.
L3 (Multi-File)	Complex fixes spanning multiple files or functions.	Refactoring auth logic, deep data sanitization.
L4 (Manual)	High-risk, semantics-heavy, or low-confidence findings.	Architecture flaws, business logic errors.

14. Agentic Remediation Loop

DevSecure v4.0 abandons the single-LLM generation model in favor of a **Multi-Agent Debate Architecture**. No single model is trusted to write and approve its own code.

14.1 The Agents

- **Patch Author Agent (Qwen3-Coder 30B-A3B):** Responsible for generating the initial fix proposal based on the finding and RAG context.
- **Primary Reviewer Agent (Qwen3-Coder):** A separate instance that critiques the proposal. It checks for correctness, style, and security regressions. It provides iterative feedback to the Author.
- **Diversity Reviewer (Different Model Family):** A conditionally invoked agent (e.g., Llama 3.1 or DeepSeek V3) triggered in high-risk scenarios (<15% of cases). Its purpose is to break "correlated hallucinations" where models from the same family share the same blind spots.

14.2 Bounded Loops

The interaction between Author and Reviewer is policy-controlled. The loop is bounded to a maximum of 1-2 iterations to prevent infinite "debate cycles." If consensus is not reached within the budget, the system fails closed and discards the fix.

15. RAG-First Knowledge System

To prevent model drift, DevSecure adopts a RAG-First (Retrieval-Augmented Generation) approach. We dynamically inject:

- Up-to-date Secure Coding Patterns (CWE-indexed).
- Repository-specific conventions (e.g., "Use `logger.error` not `fmt.Println`").
- Verified patch exemplars from the **FreshFix** pipeline.

This context is fed into the Qwen3-Coder context window to ground the generation in reality, reducing the likelihood of generic or hallucinated code.

16. Deterministic Verification Gates

Invariant: AI Never Has Approval Authority.

The "Validation Harness" of v3.0 has been replaced by a rigorous system of Deterministic Verification Gates. These gates are the only components with the authority to promote a fix to production.

16.1 Tier A Gates (Fast / Per-Iteration)

These run inside the agentic loop to provide immediate feedback:

- **Syntax/Build:** Must compile/parse.
- **Security Regression:** The *original detector* that found the bug is re-run. It must confirm the finding is gone.
- **Minimal Diff:** Enforces constraints on line count and scope to prevent "rewrite" hallucinations.

16.2 Tier B Gates (Heavy / Final PR)

These run before a PR is opened:

- **Full Test Suite:** Project unit/integration tests must pass.
- **Semantic Preservation Gate:** An AST-based analysis checks that critical semantics (authentication calls, authorization checks) were not accidentally removed.

Fail-Closed by Design: If any gate fails and the iteration budget is exhausted, the fix is discarded. We do not expose broken code to developers.

17. Graph Prioritization & Hotspots

DevSecure moves beyond list-based management to Graph-Based Risk Analysis. By constructing a dependency and call graph of the application, we identify "Hotspots"—nodes where risk is concentrated. We can tell a Technical Director: "Fixing these 3 specific files will eliminate 70% of your accumulated risk score."

18. Model Deployment on Vertex AI

Models are deployed as endpoints on Google Vertex AI for scalability and management.

- **Execution Plane:** Qwen3-Coder 30B-A3B (High performance, logic generation).
- **Classification Plane:** Qwen2.5-Coder-1.5B (Low latency, routing).

This architecture allows independent scaling: the cheap Classification Plane handles the firehose of alerts, while the expensive Execution Plane is reserved for the complex fix generation.

19. Drift & Upgrade Strategy

Software frameworks change rapidly. To manage drift:

- **Dataset Versioning:** All training data is immutably versioned.
- **Monitoring:** We track patch acceptance rates. A drop triggers investigation.
- **Blue-Green Rollout:** New models are deployed alongside old ones to verify performance before switching traffic.

20. CVEfixes -> MoreFixes -> FreshFix Evolution

Context: The CVEfixes dataset is stale. MoreFixes is better but tops out in early 2024. Strategy: We use MoreFixes as the reliable "Phase 1" baseline. "Phase 2" introduces FreshFix—our proprietary pipeline to mine data newer than 2024.

21. Phase 1 Architecture (Current State)

Currently operational. The RPS API is live. Detection is handled by Semgrep (Low CCN) and Qwen3-Coder (High CCN). Remediation utilizes the Agentic Remediation Loop with Qwen3-Coder. Classification is handled by Qwen2.5-Coder-1.5B.

22. Phase 2 GHSA Mining Pipeline (FreshFix)

Goal: Weekly data freshness. Mechanism: Ingest GHSA daily, mine diffs, apply quality gates, and append to the Gold dataset for continuous learning.

23. Security & Governance

Data Residency: EU (London). Privacy: No customer code is stored long-term without explicit consent (Training Opt-In). Secrets: Managed via Google Secret Manager. Compliance: Roadmap includes SOC 2 Type II and GDPR compliance.

24. Performance & Scaling

RPS API: <100ms p95 latency. Rate limited to 1000 req/min per tenant. Semgrep: Parallelized per-file scanning (<30s per 10k LOC). Inference: Vertex AI autoscales based on queue depth.

25. Failure Modes & Safeguards

The system is designed to be **Fail-Closed** at every layer.

- **Bounded Iteration:** To prevent infinite loops or runaway costs, the Agentic Loop has hard limits on iterations.
- **Deterministic Gates:** Unsafe fixes are caught by Tier A/B gates and discarded before reaching the user.
- **API Failure:** Fallback to cached RPS scores if the API is unreachable.

26. Competitive Technical Differentiation

Competitor Weakness	DevSecure Advantage
Snyk (Dependency focused, basic CVSS)	**Custom code focus + RPS Risk Engine**
Checkmarx (Slow, manual triage)	**AI-augmented triage + Autofix**
Aikido (Consolidation play, flat lists)	**Graph prioritization + Hotspots**
Single-LLM Remediation	**Multi-Agent Debate (Author + Reviewer + Diversity)**
Manual / "Human in the loop"	**Deterministic Authority (Automated Tier A/B Gates)**

27. IP & Defensibility

1. **RPS Risk Engine:** The specific multiplicative weighting and correlation formula integrating 9 distinct signals.
2. **FreshFix Pipeline:** The automated ability to harvest and verify new training data weekly.
3. **Multi-Agent Debate Architecture:** The specific orchestration of Patch Author, Primary Reviewer, and Diversity Reviewer.
4. **Deterministic Verification Gates:** The proprietary implementation of Semantic Preservation and Security Regression gates.
5. **26-Dimensional Routing:** The complexity vector algorithm that routes findings to L1-L4 lanes.

28. Advisor Pressure-Test Areas

- **Label Noise:** Verification of GHSA-to-Commit linkage quality.
- **Hallucination Control:** Efficacy of RAG constraints in high-complexity scenarios.
- **Agentic Loop Convergence:** Monitoring how often the Author and Reviewer reach consensus within the budget.
- **Diversity Reviewer Thresholds:** Tuning the risk triggers that invoke the expensive Diversity model.
- **Semantic Preservation:** Ensuring the AST analysis correctly identifies authentication bypass attempts.

29. Roadmap

- **Q1 2026:** Phase 1 Live. Agentic Remediation Loop operational.
- **Q2 2026:** Diversity Reviewer GA. FreshFix Pipeline Launch.
- **Q3 2026:** Semantic Preservation Gate implementation. Advanced Tier B validation suites.
- **Q4 2026:** Full autonomous remediation for L1-L3 lanes. SOC 2 Type II compliance.

30. Summary for Architects & Investors

For Technical Directors

DevSecure v4.0 is not just a tool; it is a safe autonomous agent. We have moved beyond simple "AI suggestions" to a rigorous architecture where **Fail-Closed** is the default. The Multi-Agent Debate ensures code is critiqued before it is compiled, and Deterministic Verification Gates ensure that AI never has the final say. We do not ask you to trust the model; we ask you to trust the gates.

For AI Angel Investors

Our defensibility lies in the integration of Agentic Workflows with Deterministic Authority. While competitors race to use the "best" model, we have built a moat around *how* models interact. The Multi-Agent architecture, combined with the proprietary 26-dimensional routing and FreshFix data pipeline, creates a compound learning system that gets smarter and safer with every iteration.

For Architecture Engineers

This is a system built on invariants. The separation of the Execution Plane (Qwen3) from the Classification Plane (Qwen2.5) optimizes for cost and latency. The use of Diversity Reviewers breaks the curse of correlated hallucinations. Most importantly, the architecture respects the "Fail-Closed" principle: if the agents cannot agree, or if the gates cannot verify, the system does nothing. It is safe by design.

Bottom Line: DevSecure v4.0 represents autonomous, fail-closed SAST remediation where AI proposes, AI critiques, but determinism decides.